

SplitGeo — Reporte de Desarrollo

Materia: Sistemas Georeferenciados

Institución: Universidad La Salle Bajío

Alumnos: Diego Sebastian Rojas y Cristopher Rojas Garcia

Repositorio: <https://github.com/DiegoRojas8509/Split-Geo>

Despliegue (Vercel): <https://split-geo.vercel.app>

 Logo La Salle

Capturas de pantalla

Login

 Pantalla de login

Registro

 Pantalla de registro

Dashboard — Mis grupos

 Dashboard

Detalle de grupo — Gastos y balances

 Gastos y balances

Formulario de gasto con ubicación

 Formulario de gasto

Mapa del grupo con marcadores y zonas

 Mapa

Tabla de ubicaciones

 Tabla de ubicaciones

Mapa global

 Mapa global

Swagger UI — Documentación de la API

 Swagger

1. Introducción

SplitGeo es una aplicación web de división de gastos compartidos, inspirada en Splitwise, con integración de mapas georeferenciados. Permite a grupos de usuarios registrar gastos, calcular deudas automáticamente y marcar en un mapa la ubicación donde ocurrió cada gasto. El proyecto fue desarrollado con una arquitectura fullstack separada: backend en Express.js y frontend en React con Vite, ambos desplegados en la nube.

2. Tecnologías utilizadas

Backend

Tecnología	Versión	Uso
Node.js	v20	Entorno de ejecución
Express.js	v5	Framework HTTP
MongoDB Atlas	—	Base de datos NoSQL en la nube
Mongoose	v9	ODM para MongoDB
JSON Web Token (jsonwebtoken)	v9	Autenticación stateless
bcryptjs	v3	Hash de contraseñas
swagger-jsdoc + swagger-ui-express	v6 / v5	Documentación interactiva de la API
uuid	v14	Generación de tokens de invitación
dotenv	v17	Variables de entorno
nodemon	v3	Recarga en desarrollo

Frontend

Tecnología	Versión	Uso
React	v19	Librería de interfaz
Vite	v8	Bundler y servidor de desarrollo
Tailwind CSS	v4	Estilos utilitarios
Axios	v1	Cliente HTTP
react-leaflet + Leaflet	v5 / v1.9	Mapas interactivos (OpenStreetMap)
@geoman-io/leaflet-geoman-free	v2	Trazado de zonas en el mapa
lucide-react	—	Iconografía
Photon (Komoot) API	—	Geocodificación / autocompletado de lugares

Despliegue

Servicio	Capa
Vercel	Frontend (SPA React)
Render	Backend (API Express)
MongoDB Atlas	Base de datos

3. Arquitectura del sistema



Capas del backend

El backend sigue una arquitectura estrictamente en capas:

- middleware/** — Funciones transversales: `auth.js` (verificación JWT y extracción de `req.userId`), `errorHandler.js` (captura global de errores con `next(err)`), `swagger.js` (configuración de Swagger UI en `/api-docs`).
- routes/** — Enrutadores delgados: solo reciben la petición y delegan al servicio correspondiente. No contienen lógica de negocio.
- services/** — Toda la lógica de negocio y operaciones con la base de datos. Cada entidad tiene su propio archivo de servicio.

- **models/** — Esquemas Mongoose puros, sin lógica adicional.

4. Modelos de datos

User (**splitgeo_users**)

```
{
  name:      String (required),
  email:     String (required, unique),
  passwordHash: String (required),
  timestamps: true
}
```

Group

```
{
  name:      String (required),
  description: String,
  owner:     ObjectId → User,
  members:   [ObjectId → User],
  inviteToken: String (unique)
}
```

Trip

```
{
  name:      String (required),
  description: String,
  group:     ObjectId → Group,
  createdBy: ObjectId → User
}
```

Expense

```
{
  title:      String (required),
  amount:     Number (required),
  group:     ObjectId → Group,
  paidBy:     ObjectId → User,
  splitAmong: [{ user: ObjectId, share: Number, settled: Boolean }],
  timestamps: true
}
```

Location

```
{
  name:      String (required),
  description: String,
  type:      'point' | 'zone',
  lat:       Number,
  lng:       Number,
  coordinates: [[Number]], // vértices del polígono (zona)
  group:      ObjectId → Group,
  linkedExpense: ObjectId → Expense,
  createdBy:  ObjectId → User,
  timestamps: true
}
```

5. API REST — Endpoints principales

La documentación interactiva completa está disponible en:

<https://split-geo.onrender.com/api-docs> (Swagger UI)

Método	Endpoint	Descripción	Auth
POST	/api/auth/register	Registro de usuario	—
POST	/api/auth/login	Inicio de sesión, retorna JWT	—
GET	/api/groups	Listar grupos del usuario	✓
POST	/api/groups	Crear grupo	✓
GET	/api/groups/:id	Detalle de grupo	✓
PUT	/api/groups/:id	Editar grupo	✓
DELETE	/api/groups/:id	Eliminar grupo	✓
POST	/api/groups/join/:token	Unirse a grupo por enlace	✓
GET	/api/trips/group/:groupId	Listar viajes de un grupo	✓
POST	/api/trips	Crear viaje	✓
PUT	/api/trips/:id	Editar viaje	✓
DELETE	/api/trips/:id	Eliminar viaje	✓
GET	/api/expenses/group/:groupId	Listar gastos de un grupo	✓
POST	/api/expenses	Crear gasto con división	✓
PUT	/api/expenses/:id	Editar gasto	✓
DELETE	/api/expenses/:id	Eliminar gasto	✓

Método	Endpoint	Descripción	Auth
PATCH	<code>/api/expenses/:id/settle</code>	Marcar deuda como liquidada	✓
GET	<code>/api/locations/group/:groupId</code>	Ubicaciones de un grupo	✓
GET	<code>/api/locations/group/:groupId?name=X</code>	Búsqueda por nombre	✓
GET	<code>/api/locations</code>	Todas las ubicaciones	✓
POST	<code>/api/locations</code>	Crear ubicación (punto o zona)	✓
PUT	<code>/api/locations/:id</code>	Editar ubicación	✓
DELETE	<code>/api/locations/:id</code>	Eliminar ubicación	✓

6. Funcionalidades implementadas

6.1 Autenticación — Login y Registro

El sistema de autenticación utiliza JWT (JSON Web Tokens) con contraseñas hasheadas mediante `bcryptjs`.

- **Registro:** El usuario proporciona nombre, correo y contraseña. El backend valida unicidad del correo y almacena el hash de la contraseña. Retorna un token JWT.
- **Login:** Verifica correo y contraseña. Si son correctos, retorna un token JWT que el frontend guarda en `localStorage`.
- **Protección de rutas:** El middleware `auth.js` extrae el token del header `Authorization: Bearer <token>` y adjunta el `userId` a cada petición autenticada. Las rutas no autenticadas redirigen al login.

6.2 CRUD 1 — Grupos

Los grupos son el contenedor principal de la aplicación. Cada grupo puede tener múltiples miembros invitados mediante un enlace único.

- **Crear:** Nombre y descripción opcionales. Al crear, se genera automáticamente un `inviteToken` único (UUID).
- **Leer:** Lista de todos los grupos donde el usuario es miembro o propietario.
- **Editar:** Nombre y descripción del grupo (solo el propietario).
- **Eliminar:** Solo el propietario puede borrar el grupo.
- **Unirse:** Cualquier usuario autenticado puede unirse a un grupo mediante el enlace de invitación (`/join/:token`).

6.3 CRUD 2 — Viajes / Eventos

Dentro de cada grupo se pueden crear viajes o eventos para organizar los gastos.

- **Crear:** Nombre y descripción.
- **Leer:** Lista de viajes del grupo.
- **Editar:** Nombre y descripción.
- **Eliminar:** El creador puede eliminar el viaje.

6.4 Gestión de Gastos

El módulo de gastos es el núcleo de la aplicación:

- Registro de un gasto con título, monto total y quién pagó.
- División del gasto entre los miembros seleccionados (partes iguales o montos personalizados).
- Validación visual: el formulario muestra en tiempo real si los montos divididos suman el total.
- **Cálculo automático de balances:** El sistema calcula quién le debe a quién y cuánto, mostrando:
 - Total que te deben y número de personas
 - Total que tú debes y a cuántas personas
 - Detalle individual por persona (nombre + monto)
- Posibilidad de **liquidar** la deuda propia en cada gasto.
- Edición y eliminación de gastos.

6.5 Registro de ubicaciones en el mapa

Cada gasto puede tener una ubicación geográfica asociada:

- **Autocompletado de lugares:** El campo de ubicación dentro del formulario de gasto utiliza la API de Photon (OpenStreetMap), que entrega sugerencias en tiempo real mientras el usuario escribe (mínimo 2 caracteres, debounce de 300ms). Los resultados se priorizan geográficamente por cercanía a Medellín, Colombia.
- Al seleccionar una ubicación, se almacena en MongoDB con nombre, latitud y longitud, vinculada al gasto y al grupo.
- El pin aparece automáticamente en el mapa del grupo.
- El mapa hace animación **flyTo** hacia el lugar seleccionado.

6.6 Edición de ubicaciones desde el popup

Al hacer clic sobre un marcador en el mapa, aparece un popup con:

- Nombre del lugar
- Descripción
- Coordenadas (lat, lng)
- Botón **Editar** → abre modal con formulario prellenado
- Botón **Borrar** → elimina el punto del mapa y la base de datos

6.7 Eliminación de ubicaciones

Las ubicaciones se pueden eliminar:

- Desde el popup del marcador en el mapa.
- Automáticamente al eliminar el gasto al que están vinculadas (cascade delete en el backend).

6.8 Tabla de ubicaciones en tiempo real

La tabla muestra todos los puntos registrados del grupo con las columnas:

- **Nombre** — Nombre del lugar
- **Descripción** — Descripción asociada

- **Latitud** — Coordenada geográfica
- **Longitud** — Coordenada geográfica
- **Creado por** — Nombre del usuario

La tabla se actualiza en tiempo real tras cada operación de alta, edición o borrado. Al hacer clic en cualquier fila, el mapa hace animación **flyTo** hacia ese punto.

6.9 Búsqueda de ubicaciones por nombre

Encima de la tabla hay un campo de búsqueda que realiza una consulta al endpoint **GET** `/api/locations/group/:id?name=X`. El backend filtra con expresión regular case-insensitive (`$regex, $options: 'i'`), permitiendo búsquedas parciales.

6.10 Trazado de zonas / polígonos

El mapa incluye controles de dibujo mediante la librería **@geoman-io/leaflet-geoman-free**:

- Herramienta de **polígono libre** y **rectángulo**.
- Al terminar de trazar, aparece un modal que pide nombre y descripción de la zona.
- La zona se persiste en MongoDB como **type: 'zone'** con el array de coordenadas de sus vértices.
- Al recargar la página, las zonas se renderizan nuevamente en el mapa como polígonos indigo semitransparentes.
- Cada zona tiene popup con nombre, descripción y botón para eliminar.

6.11 Vistas de la aplicación

La aplicación cuenta con las siguientes vistas diferenciadas:

Vista	Ruta	Descripción
Login	<code>/login</code>	Formulario de inicio de sesión
Registro	<code>/register</code>	Formulario de alta de usuario
Dashboard	<code>/</code>	Lista de grupos del usuario con opción de crear/editar/borrar
Detalle de grupo	<code>/groups/:id</code>	Gastos, balances, mapa, tabla y búsqueda de ubicaciones
Unirse a grupo	<code>/join/:token</code>	Ruta de invitación, une al usuario y redirige
Mapa global	<code>/map</code>	Todas las ubicaciones de todos los grupos

7. Despliegue

Arquitectura de despliegue




```

split-geo.vercel.app
VITE_API_URL → Render
|
▼
Render (Backend)
split-geo.onrender.com
Variables de entorno:
- MONGO_URI
- JWT_SECRET
- PORT
|
▼
MongoDB Atlas
cluster: practicamongo
base de datos: geoexpress

```

Proceso de despliegue

Backend (Render):

1. Conectar repositorio GitHub al servicio de Render.
2. Root Directory: `backend`
3. Build Command: `npm install && npm run build` (compila el frontend también)
4. Start Command: `npm start`
5. Configurar variables de entorno `MONGO_URI`, `JWT_SECRET`, `PORT`.

Frontend (Vercel):

1. Importar repositorio en Vercel.
2. Root Directory: `frontend`
3. Build Command: `npm run build`
4. Output Directory: `dist`
5. Variable de entorno: `VITE_API_URL=https://split-geo.onrender.com/api`

8. Seguridad

- Las contraseñas nunca se almacenan en texto plano; se hashean con `bcryptjs` (salt rounds = 10).
- El JWT se firma con una clave secreta de 64 caracteres y expira en 7 días.
- El archivo `.env` está excluido del repositorio mediante `.gitignore`.
- Las rutas protegidas verifican el token en cada petición mediante el middleware `auth.js`.
- La colección de usuarios utiliza el nombre `splitgeo_users` para evitar colisiones con otras colecciones en la base de datos compartida de práctica.

9. Conclusiones

SplitGeo cumple con todos los criterios establecidos en la rúbrica del proyecto final:

Criterio	Implementación	Puntos
Login	JWT + bcryptjs, página /login	10
Registro	Validación de email único, página /register	10
CRUD interno 1	Grupos: crear, leer, editar, eliminar	5
CRUD interno 2	Viajes: crear, leer, editar, eliminar	5
Registro de ubicación en mapa	Autocompletado Photon + pin en mapa + guardado en MongoDB	5
Edición de ubicación (popup)	Modal desde popup del marcador	5
Eliminación de ubicación (popup)	Botón en popup del marcador	5
Tabla de ubicaciones en tiempo real	Tabla actualizada tras cada operación	10
Búsqueda por nombre	GET /locations/group/:id?name=X + componente de búsqueda	10
Trazado de zonas	leaflet-geoman-free → persistencia en MongoDB	15
3+ vistas	Login, Register, Dashboard, GroupDetail, MapPage, JoinGroup	10
Backend por capas	middleware/ + routes/ + services/ + models/	10
Total		100

El proyecto integra de manera coherente la gestión financiera colaborativa con la georeferenciación, permitiendo a los usuarios no solo dividir gastos sino también visualizar geográficamente dónde ocurrieron, lo que añade valor tanto funcional como educativo al sistema.